

AI Resume Screener

Full Project Roadmap — Spring Boot · React · PostgreSQL · Claude API

Build and deploy a full-stack AI tool that matches a resume (PDF) against a job description and returns a structured match score, skill gaps, and suggested edits — powered by Claude API. Target: live URL within 5 weeks.

Tech Stack

Layer	Technology	Why
Backend	Spring Boot 3.x (Java 17)	Your primary skill, great for REST APIs
Frontend	React 18 + Vite	Lightweight, fast setup
Database	PostgreSQL	Structured data, relational queries
AI	Claude API (claude-sonnet-4-6)	Best for structured JSON output
PDF Parsing	Apache PDFBox	Extracts text from uploaded resumes
Auth	Spring Security + JWT	Industry standard
Deploy	Railway + Vercel	Free, beginner-friendly

Architecture Overview

[illegible]

Database Schema

```
users → id, email, password_hash, created_at
resumes → id, user_id, file_name, extracted_text, uploaded_at
analyses → id, user_id, resume_id, job_description,
match_score, skill_gaps (jsonb), strengths (jsonb),
suggestions (jsonb), created_at
```

Build Phases

Phase 1 — Project Setup

Day 1 · ~2 hours

Backend — generate at start.spring.io with these dependencies:

- Spring Web, Spring Security, Spring Data JPA, PostgreSQL Driver, Lombok, Validation

application.yml:

```
spring:
  datasource:
    url: ${DATABASE_URL}
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate:
      ddl-auto: update
  claude:
    api-key: ${CLAUDE_API_KEY}
  model: claude-sonnet-4-6
```

Frontend setup:

```
npm create vite@latest resume-screener-ui -- --template react
cd resume-screener-ui
npm install axios react-router-dom
```

Phase 2 — Auth (JWT)

Day 1–2 · ~3 hours

Endpoints to build:

```
POST /api/auth/register { email, password } → { token }
POST /api/auth/login { email, password } → { token }
```

What to build:

- User entity + UserRepository
- AuthController → AuthService → JwtUtil
- JwtAuthFilter added to Spring Security filter chain
- SecurityConfig — permit /api/auth/**, secure everything else
- React: Login + Register pages, store JWT in localStorage, Axios interceptor

Add to pom.xml:

```
org.apache.pdfbox
pdfbox
3.0.1
```

PDFParser utility:

```
public String extractText(MultipartFile file) throws IOException {
    try (PDDocument doc = Loader.loadPDF(file.getBytes())) {
        PDTextStripper stripper = new PDTextStripper();
        return stripper.getText(doc);
    }
}
```

Endpoints:

```
POST /api/resumes/upload multipart/form-data (PDF) → { resumeId, extractedText }
GET /api/resumes → list of user's resumes
```

Phase 4 — Claude API Integration

Day 3–4 · ~4 hours

This is the core of the app. You will call Claude with a carefully crafted prompt and parse its structured JSON response. Invest time in the prompt — it directly determines output quality.

The prompt (copy this exactly to start):

```
You are an expert resume reviewer and recruiter.

Analyze the match between the resume and job description below.
Respond ONLY with valid JSON in this exact format:

{
  "match_score": ,
  "strengths": ["strength 1", "strength 2"],
  "skill_gaps": ["missing skill 1", "missing skill 2"],
  "suggestions": [
    { "original": "existing bullet", "improved": "suggested rewrite" }
  ],
  "summary": "2-3 sentence overall assessment"
}

RESUME: {{resumeText}}
JOB DESCRIPTION: {{jobDescription}}

Respond with JSON only. No explanation, no markdown, no backticks.
```

Analysis endpoint:

```
POST /api/analysis/run
Body: { resumeId: 1, jobDescription: "paste JD here" }
→ { analysisId, matchScore, strengths, skillGaps, suggestions, summary }
```

Phase 5 — History + Dashboard

Day 4–5 · ~2 hours

```
GET /api/analysis/history → all analyses for logged-in user
GET /api/analysis/{id} → single analysis detail
```

React dashboard:

- List of past analyses with score badge, resume name, JD snippet, date
- Click any row → full detail view: strengths, gaps, suggestions side by side
- Score badge: green >70%, amber 40–70%, red <40%

Phase 6 — Polish Before Deployment

Day 5–6 · ~3 hours

- Input validation — max 5MB file size, .pdf only, JD min 100 characters
- Global error handler — @ControllerAdvice returning { error, message } JSON
- Rate limiting — max 5 analyses per user per day (prevents surprise API bills)

- Loading states in React — spinner while Claude responds (3–8 seconds)
- CORS config — whitelist your Vercel URL in Spring Security
- README — architecture diagram, live URL, screenshots, setup steps

Deployment

Step 1 — Push to GitHub

```
git add .
git commit -m "feat: complete AI resume screener MVP"
git push origin main

# Verify: no API keys or passwords in committed files
# application.yml must only use ${ENV_VAR} placeholders
```

Step 2 — Deploy PostgreSQL + Backend on Railway

1. Go to railway.app → New Project → Deploy from GitHub repo
2. Railway auto-detects Spring Boot (looks for pom.xml)
3. Add PostgreSQL plugin — Railway auto-injects DATABASE_URL
4. In Variables tab, add:

```
CLAUDE_API_KEY = sk-ant-...
DB_USER = postgres
DB_PASS =
DATABASE_URL =
```

5. Railway builds your JAR and gives you a public URL

Test it:

```
curl https://your-app.up.railway.app/api/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"test@test.com","password":"test123"}'
```

Step 3 — Deploy React on Vercel

1. Go to vercel.com → New Project → Import GitHub repo (frontend)
2. Vercel detects Vite automatically
3. Add environment variable:

```
VITE_API_BASE_URL = https://your-app.up.railway.app
```

4. In your React code:

```
const API = import.meta.env.VITE_API_BASE_URL;
```

Step 4 — Fix CORS in Spring Boot

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://your-project.vercel.app"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);
    return source;
}
```

Push → Railway auto-redeploys. Your app is now live.

Resume Bullets (add after going live)

Use real numbers — run a few analyses and note how long Claude takes, or estimate based on manual review time saved. Metrics matter more than tech names.

"Built and deployed a full-stack AI resume screening tool using Spring Boot, React, and PostgreSQL; integrated Claude API to generate structured match scores and skill-gap reports from resume-JD pairs."

"Implemented a prompt-engineering pipeline using Apache PDFBox for text extraction and Claude API for JSON-structured analysis, reducing manual resume review effort significantly."

"Deployed backend on Railway with PostgreSQL and frontend on Vercel; configured JWT auth, CORS, and environment-based secrets management for production readiness."

Week-by-Week Schedule

Week	Goal	Done when...
Week 1	Phase 1 + 2 — Setup + Auth	Register/login works, JWT returned
Week 2	Phase 3 — PDF Upload + Parsing	PDF uploads, text saved to DB
Week 3	Phase 4 — Claude Integration	Analysis endpoint returns JSON from Claude
Week 4	Phase 5 — History + Dashboard	User can see all past analyses
Week 5	Phase 6 + Deployment	Live URL works end-to-end

Each week assumes ~6–8 hours of focused work (3 weekday evenings or one solid weekend day).

Things to Keep in Mind

- Commit often — at least one commit per feature. Employers look at commit history.
- Never commit your Claude API key — use .env locally, Railway variables in prod.
- Write a proper README — live URL, one screenshot, tech stack. First thing a recruiter opens.

- Add a demo user — seed `demo@example.com / demo123` so anyone can try it without registering.
- Keep the Claude bill in check — add a daily limit of 5 analyses per user from day one.